

Software Engineering

Books or notes are **not** allowed.

Write only on these sheets. **Concise** and **readable** answers please.

Surname, name, matricola _____

Bus information

Consider a metropolitan area with many bus lines and bus stops. A display at each bus stop shows the estimated waiting time for all the bus lines stopping there. Besides, a user can obtain the same information sending an sms to a defined number, with the ID of the bus stop.

In the following you should analyze and model the application that provides waiting time information for buses.

1 – a. Define the **context diagram** (including relevant interfaces)

Actor	Physical interface	Logical interface
User	Smartphone, display at bus stop	Request via sms, waiting time info on display, on sms
Bus company admin	PC	Set up (bus lines, bus stops), associate bus to bus line



1-b Define the **glossary** (key concepts and their relationships) (UML class diagram) for the application

A bus line is a sequence of bus stops (ex line 13 stops in bus stops 10, 3 and 24). A bus serves on a certain bus line and at each time has a certain position. The bus line defines the path a bus must follow, and has schedules with estimated arrival times at each bus stop

(ex for line 13

 schedule1 is bus stop 10-9.00am

 bus stop 3-9.10am

 etc

 schedule2 is bus stop 10-9.10am

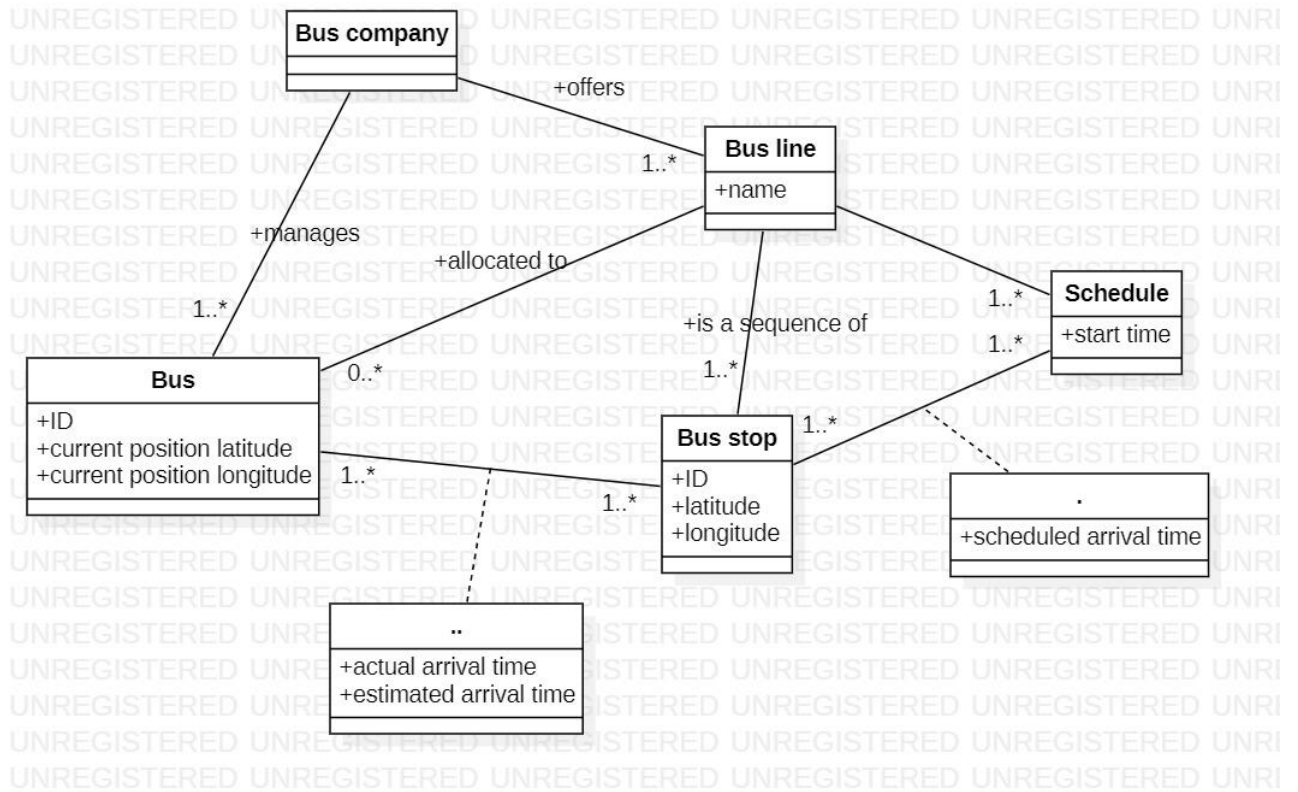
 bus stop 3-9.200am

 etc

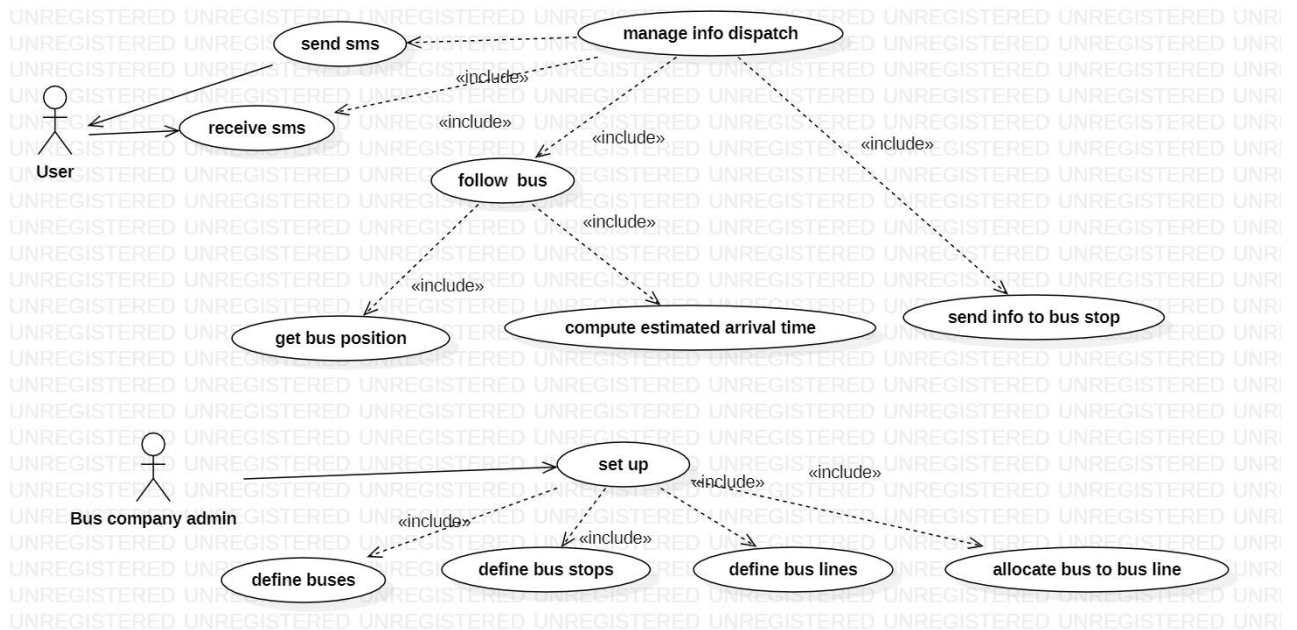
)

As the bus moves, there can be delays, using the current position of the bus (from the gps on the bus) and distance from next bus stop, it is possible to compute the estimated arrival time = scheduled arrival time + delay.

Collecting the actual arrival times of each bus it is possible to obtain all delays, and possibly modify the schedule.

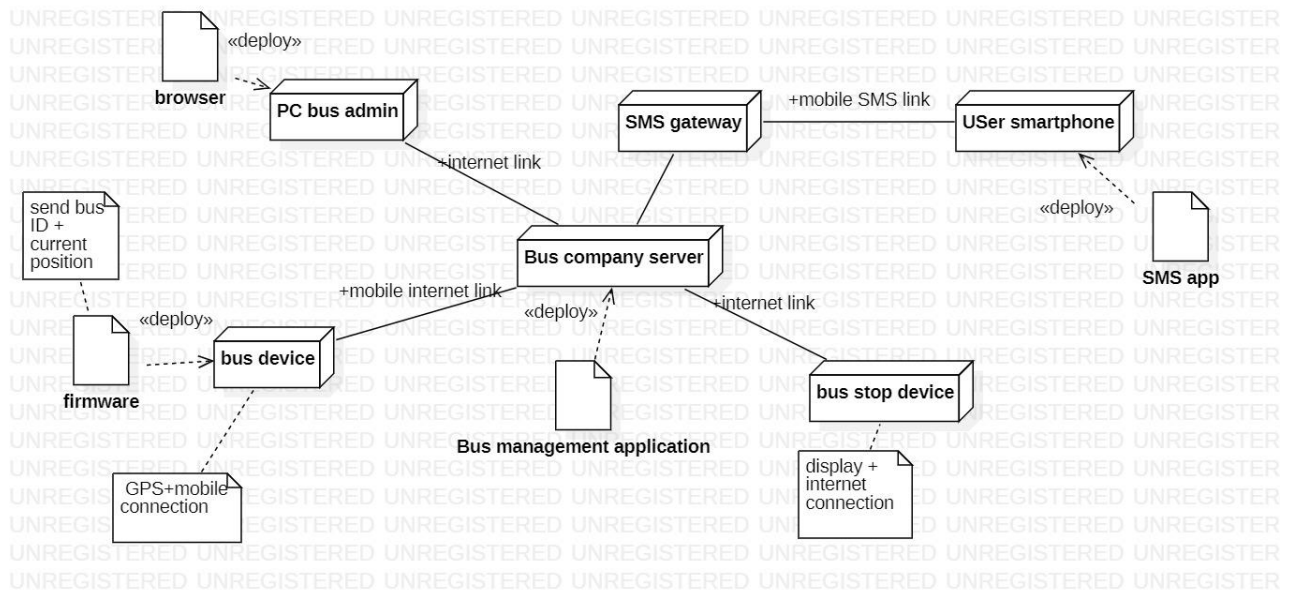


1-c Draw the Use Case Diagram for the application. For each Use Case use long self explaining names, or give apart a short description of what they do



1-d Draw the deployment diagram

It is a classic client (displays at bus stops) server (back end server of bus company) system. All info on buses (positions) is concentrated at server, that has also all info on bus stops, and bus lines. A peer to peer model (each bus stop has info on other bus stops, and buses, and bus lines) is way more complicated, duplicates information (bus tops, bus lines, schedules, bus positions), requires more expensive hardware at bus stops, generates more network traffic. Besides, hardware at bus stops is subject to damage and theft, so it should be as cheap as possible. With the client server approach each bus stop can be equipped with a simple display with minimum processing capability (receiving html page from an internet link, display page - less than 100 euro).



2 -Define black box tests for the following function, using equivalence classes and boundary conditions. The computes if a child has priority for a transplant

boolean hasPriority(int birthDay, int insertionDay)

Input: birthday, birthday of the child, expressed as number of days since 1 1 2000
insertionDay, day when the transplant was asked for, also expressed as number of days since 1 1 2000

Output: TRUE if the child is aged less than 2 years (by definition 730 days)

AND

The transplant was asked for when the child was less than one year old (365days)

FALSE otherwise

Remarks: the tests should be performed today (6 sep 2024 or day 9015 from 1 1 2000)

Ex. hasPriority(4383, 4384) → F (or born 1 1 2012 and transplant asked the day after)

hasPriority(3653, 3654) → F (or born 1 1 2010 and transplant asked the day after)

Each row in the table represents an equivalence class. Total number of equivalence classes = $3 \times 3 \times 3 \times 2 = 54$ For this reason some combinations are pruned at the first invalid range encountered. For the same reason boundary test cases are defined around one equivalence class only (see T_b test cases).

<i>birthDay</i>	<i>insertionDay</i>	<i>Birthday vs insertion Day</i>	<i>Child Age</i>	<i>Valid Invalid</i>	<i>Test case</i>
[minint, 0[[0, 9015] [9016, maxint]	[minint, 0[[0, 9015] [9016, maxint]	insertionDay < birthDay birthDay <= insertionDay <= birthDay+365 insertionDay > birthDay+365	Age <= 2 years Age > 2 years		
[minint, 0[	-	-	-	I	T(-3,1000) → err
[0, 9015]	[minint, 0[	-	-	I	T(1000,-3) → err
	[0, 9015]	insertionDay < birthDay	-	I	T(900,1000) → err
		birthDay <= insertionDay <= birthDay+365	Age <= 2 years	V	T(9000,9001) → TRUE Boundaries: (birthDay=insertion

					Day) Tb(9000,9000)→ TRUE (insertionDay= birthday+1) Tb(9000,9001)→ TRUE (insertionDay= birthday+365) Tb(8000,8365) → TRUE (insertionDay= birthday+366) Tb(8000,8366) → FALSE (insertionDay= birthday+364) Tb(8000,8364) → TRUE
			Age > 2 years	V	T(3653,3654) → FALSE
		insertionDay > birthDay+365	Age <= 2 years	V	T(8000,8366) → FALSE
			Age > 2 years	V	T(3653,4474) → FALSE
	[9016, maxint]	-	-	I	T(9000,11000)→ Err
[9016, maxint]	-	-	-	I	T(10000,3650)→ Err

3 – For the following function define the control flow graph, and define test cases to obtain the highest possible node coverage, edge coverage, multiple condition coverage, loop coverage, path coverage.

For the test cases, **write only the input value.**

```
public int penalties(boolean[] penaltyTeam1, boolean[] penaltyTeam2){
1          int scoreTeam1= 0, scoreTeam2 = 0 , winner=0;
2          for (int i=0;i <= 4; i++){
3              if (penaltyTeam1[i])
4                  scoreTeam1++;
5              if (penaltyTeam2[i])
6                  scoreTeam2++;
7          }
8          if (scoreTeam1 > scoreTeam2)
9              winner = 1;
10         else if (scoreTeam2 > scoreTeam1)
11             winner = 2;
12         else
13             winner = 0;
14         return winner;
15}
```

Coverage type	Number of test cases needed to obtain 100% coverage	Coverage obtained with test cases defined (%)	Test cases defined
statement	3	100	T1 ([T,T,T,T,T], [T,F,T,T,T]) T2 ([T,F,T,T,T], [T,T,T,T,T]) T3 ([T,T,T,T,T], [T,T,T,T,T])
decision	3	100	T1 T2 T3
Loop	3	1/3	With any input the for will loop 4 times
Path	*	*	

*The possible paths in loop are the total number of possible combinations of penaltyTeam1 and penaltyTeam2 is $2^{10} = 1024$ combinations.

Then there are 3 more paths on node 8 to 14 (8-9-14, 10-11-14, 10-13-14).

So, the total number of paths is $1024*3 = 3072$.

In theory writing all these test cases is not impossible, however will require a long time

4 – Give the definition of calendar time in the context of project management, and its unit of measure

Calendar time is a measure of ‘duration’ (NOT effort)

5 – Describe briefly the ‘repository’ architectural style

See slides

6 – What is the recommended length of an iteration in an agile project (ex Scrum, Extreme Programming)? And why such length?

4 weeks

7 – A software product is developed custom for a bank. Development takes 18 months, operation and maintenance 10 years. Where do you expect to have the larger part of costs? And why?

Most likely maintenance, because it lasts (calendar time) circa 7 times more.